

Full-Stack Security Audit & Improvement Report

This document outlines the comprehensive security enhancements implemented across the **Backend**, **Frontend**, and **AI/ML Microservices** of the AI-Integrated E-commerce Platform.

1. Backend API Security (Node.js/Express)

The Node.js backend serves as the core API gateway and handles authentication, meaning its security is paramount. The following improvements were made to harden it against common web vulnerabilities:

Implemented Protections

- **Helmet Middleware:** Injected `helmet()` into the Express pipeline. This automatically sets 14 crucial HTTP headers (like `Strict-Transport-Security` and `X-DNS-Prefetch-Control`) to protect against well-known web vulnerabilities.
- **Data Sanitization:**
 - Added `express-mongo-sanitize` to recursively remove any keys containing `$` or `.` from request bodies, query strings, and parameters. This entirely prevents MongoDB Operator Injection attacks.
 - Added `xss-clean` to sanitize user input coming from POST bodies, GET queries, and URL params to prevent Cross-Site Scripting (XSS).
- **Authentication Rate Limiting:** Added a dedicated `express-rate-limit` to the `/login` endpoints in `authRouter.js`. This restricts a single IP address to 10 login attempts per 15 minutes, mitigating brute-force dictionary attacks against user passwords.
- **Strict CORS Enforcement:** Removed wildcard `*` domains and implemented strict Regex-based CORS origin matching, ensuring only authorized frontend clients (local and Railway production domains) can make state-modifying API requests.
- **Credential Management:** Removed hardcoded fallback plaintext passwords from the database connection logic. The system now strictly relies on the `.env` variables, preventing accidental commits of live database credentials.

2. Frontend Security (React/Vite & Nginx)

The frontend application stores sensitive JSON Web Tokens (JWTs) in the browser's `localStorage`. Because `localStorage` is vulnerable to extraction via malicious JavaScript, the primary defense is ensuring no unauthorized scripts can run.

Implemented Protections

- **Content Security Policy (CSP):** Added a strict `<meta http-equiv="Content-Security-Policy">` tag to `index.html`. This creates an allowlist of origins from which scripts, images, and styles can be loaded, neutralizing XSS injection risks even during local Vite development.
- **Nginx Hardening:** Upgraded the production `nginx.conf` file to inject security headers on every static file response served by the Docker container:
 - `X-Frame-Options "SAMEORIGIN"`: Prevents clickjacking by ensuring the site cannot be embedded in an iframe on an attacker's domain.
 - `X-XSS-Protection "1; mode=block"`: Instructs older browsers to aggressively block rendering if they detect reflected XSS.
 - `X-Content-Type-Options "nosniff"`: Prevents the browser from trying to guess the MIME type of a file, stopping malicious uploads from being executed as scripts.
 - `Referrer-Policy "strict-origin-when-cross-origin"`: Ensures the full URL (which might contain sensitive query parameters) isn't leaked to external third-party sites.

3. AI/ML Microservices (Python/FastAPI)

The platform features several AI microservices (`face-service`, `restock-ml`, `recommendation-engine`, `youtube-trending`). A critical architectural flaw was identified where these services were exposed directly to the host network without authentication.

Implemented Protections

- **Docker Network Isolation:** Adjusted the `ports` mapping in `docker-compose.yml`. Previously, they were mapped to `0.0.0.0` (all interfaces). They are now strictly bound to `127.0.0.1` (localhost).
 - *Why this matters:* This prevents external devices on the network or internet from bypassing the secure Node.js backend and querying the AI models directly (which could lead to a Denial of Service or data extraction).
 - *Developer Convenience:* Because they are mapped to localhost, developers can still easily access the FastAPI Swagger UI locally (`http://127.0.0.1:5000/docs`).

- **CORS Hardening:** Modified the `CORSMiddleware` inside the FastAPI applications (e.g., `app.py`, `src/api.py`). The overly permissive `allow_origins=["*"]` was removed and replaced with explicit origins matching the internal Docker backend network, adhering to the principle of least privilege.

Summary

The platform has successfully transitioned from an implicitly trusted architecture to a defense-in-depth model.

1. The **Backend** now actively sanitizes inputs and throttles malicious traffic.
2. The **Frontend** uses CSP and Nginx headers to prevent the browser from executing rogue scripts or falling victim to clickjacking.
3. The **AI Microservices** are now safely isolated behind the Node.js backend gateway, unreachable by external threats.